The Data Life Cycle of a GPT Model: A Technical Report on Computation, Processing, and Statistical Analysis

Part I: The Data Genome of GPT: Sourcing and Curation

A. The Grand Corpus: From Public Web to Proprietary Curation

The remarkable capabilities of a GPT-style model are fundamentally predicated on its access to vast, diverse, and meticulously curated datasets. The initial step in the model's life cycle is the acquisition of a "corpus," a large, structured collection of text and other data from which the model can learn. The scale of this data is immense, often measured in petabytes and billions of pages. For instance, the Common Crawl project, a non-profit organization, maintains a free and open repository of web crawl data that has amassed over 300 billion pages spanning 18 years.[1] Early foundational models, such as GPT-3, were heavily reliant on this single source, with over 80% of its training tokens originating from the Common Crawl dataset.[2]

However, the raw, unfiltered nature of web-scale data necessitates a profound process of refinement. The field has evolved beyond reliance on single, monolithic corpora to the development of more diverse, curated datasets. A prime example is The Pile, a dataset constructed by EleutherAI, which combines 22 smaller, high-quality datasets to enhance a model's cross-domain knowledge.[3] This curated collection includes a filtered version of Common Crawl (Pile-CC) alongside academic sources like arXiv and PubMed Central, as well as conversational data from platforms like Stack Exchange.[3] This strategic combination of sources enables models trained on The Pile to better handle a wider range of situations and

writing styles than those trained on less varied data.[3]

Despite the move toward curation, the inherent limitations and biases of raw data persist. The Common Crawl dataset, while massive, is not a faithful replica of the "entire web".[2] It exhibits a significant bias toward English-language pages and its regional coverage is inconsistent. Furthermore, an increasing number of prominent domains, including

The New York Times, actively block Common Crawl from accessing their content, which further skews the representativeness of the dataset.[2] The Mozilla Foundation has raised concerns that uncritically using Common Crawl as a proxy for the global web effectively declares a small, primarily English-speaking fraction of the internet as representative of the entire world.[2]

A more profound challenge arises from the intentional inclusion of problematic content in these raw datasets. For example, the mission of Common Crawl is to provide data for a variety of use cases, which includes research on hate speech.[2] This deliberate inclusion means that robust data filtering is not merely a beneficial step; it is an ethical imperative. However, many popular filtered versions of these datasets, such as the Colossal Clean Crawled Corpus (C4), have been found to rely on "simplistic automated filtering techniques".[2] For instance, a method of removing pages containing words from a "List of Dirty, Naughty, Obscene, and Otherwise Bad Words" may inadvertently and incorrectly flag non-toxic language used by LGBTQIA+ communities, thereby undermining their representation and creating a biased dataset.[2]

This process illustrates a critical, yet often unexamined, dynamic in data preparation. The initial data source is deliberately broad, containing content that is known to be problematic for specific applications. Subsequent data cleaning, which is supposed to mitigate these issues, is often executed with automated, blunt-force techniques that are inadequate for the task. This creates a causal chain of events where the biases and problematic content from the raw corpus are not merely inherited but can be amplified in the final refined dataset, which in turn imprints these biases onto the trained model. The discovery that the C4 dataset, despite being marketed as a "cleaner, smaller" version of Common Crawl, contained a significant amount of offensive content and disproportionately excluded content from minority groups, directly correlates with its lack of transparency regarding its filtering methodologies.[5] The challenge of data bias is thus not merely a technical problem to be solved with better algorithms, but a profound ethical and social issue rooted in the fundamental choices made during data curation.

B. The Imperative of Cleanliness: The Data Preprocessing Pipeline

Once data has been sourced, it must undergo a rigorous preprocessing pipeline to optimize the training process and improve the final model's performance. This multi-stage process is essential for removing noise, mitigating risks from toxic content, and ensuring data diversity.[6] A typical pipeline involves several critical steps: preliminary text cleaning, heuristic filtering, deduplication, and model-based quality filtering.[6]

Preliminary text cleaning focuses on rectifying basic structural and formatting issues. This involves removing or correcting irrelevant information, fixing structural errors, normalizing text (e.g., converting to lowercase), and ensuring a consistent format for punctuation and spacing.[7] Heuristic filtering then applies a more nuanced, cascading approach to enforce specific quality criteria. This can include removing documents that are too short or too long, or those with an unusual character-to-word ratio.[6]

A more advanced technique is model-based quality filtering, which leverages statistical measures to assess the linguistic quality of a text. A key metric in this context is perplexity. In natural language processing, perplexity measures how well a language model predicts a text sequence.[9] It is a measure of the model's uncertainty or confusion about the text. Mathematically, perplexity is inversely related to the sequence probability, meaning a lower perplexity score indicates a higher-quality, more probable text sequence.[9] This is because a model is more "confident" in its prediction of each token in a high-quality, linguistically sound text. Therefore, perplexity can be used as a filtering criterion to remove low-quality text from the training corpus before it is used to train the final model.[9]

The following table provides a summary of the data preprocessing stages and their objectives:

Stage	Purpose	Methods	Data Issues Addressed
Download & Extract	Access raw data from source	JSONL, CSV extraction	File formats, data structure
Preliminary Cleaning	Standardize and remove noise	Lowercasing, noise removal	Formatting, irrelevant content
Heuristic Filtering	Enforce quality criteria	Length filters, content-specific rules	Low-quality text, formatting errors
Deduplication	Eliminate redundant data	Exact, fuzzy, semantic hashing	Overfitting, computational cost

Model-Based Filtering	Refine data based on quality	Perplexity scoring, PII redaction	Linguistic quality, sensitive data
Blending & Shuffling	Create diverse training batches	Proportional blending, randomization	Data bias, training stability

The entire data preprocessing pipeline can be visualized as a sequential flow of operations:

Image: The LLM Data Preprocessing Pipeline
A flowchart illustrating the multi-stage data preprocessing pipeline. The process begins with Raw Corpus (e.g., Common Crawl), which flows into a Preprocessing Pipeline. This pipeline is comprised of a series of sequential, labeled boxes: Preliminary Text Cleaning, Heuristic Filtering, Deduplication, Model-Based Quality Filtering, and Blending & Shuffling. Each box represents a distinct stage. The final output of this pipeline is a Cleaned, Deduplicated, and Normalized Dataset, which is then used for training the Large Language Model.

C. The Science of Deduplication

A critical and computationally intensive stage of the data pipeline is deduplication. The practice is essential for improving model training efficiency, reducing computational costs, and, most importantly, ensuring data diversity and preventing overfitting.[6] Research has shown that training on deduplicated datasets is not only faster but can also result in models with similar or superior perplexity to those trained on data that has not been deduplicated.[10] The presence of redundant data can cause the model to "memorize" the repeated content rather than learning and generalizing from it, a classic form of overfitting.

The deduplication process can be implemented through three primary approaches: exact, fuzzy, and semantic.[6] Exact deduplication is the most straightforward, involving the generation of a hash signature for each document and removing all but one instance of identical documents.[6] Fuzzy and semantic deduplication are more sophisticated, designed to identify documents with minor modifications, different formatting, or conceptually similar content.[6]

A powerful method for fuzzy and semantic deduplication is the combination of MinHash and Locality-Sensitive Hashing (LSH). This approach is particularly effective at a massive scale, where traditional pairwise comparisons would be prohibitively slow and costly.[11] The process is founded on the concept of

Jaccard similarity, a measure of the overlap between two sets. For two sets, A and B, the

Jaccard similarity is defined as:

$J(A,B)=|A \cup B| \, |A \cap B|$

Where $|A \cap B|$ is the number of elements in the intersection of sets A and B, and $|A \cup B|$ is the number of elements in their union.[11]

Directly calculating the Jaccard similarity for every document pair is computationally infeasible for large corpora. MinHash provides an efficient probabilistic method to estimate this similarity. The process begins with "shingling," where each document is converted into a set of overlapping token sequences. Multiple independent hash functions are then applied to each shingle, and the minimum hash value for each function is recorded to form a compact MinHash signature.[11] The core principle of MinHash is that the more similar two documents are, the more likely their MinHash signatures will match at the same positions.[11]

Even with compact MinHash signatures, comparing every single pair is still inefficient at scale. This is where LSH comes into play. LSH is a technique that clusters similar items into the same "buckets" with a high probability, eliminating the need for exhaustive comparisons.[11] An

n-dimensional MinHash signature is divided into b bands, with each band containing r consecutive hash values, such that $n=b \times r$. Each band is then independently hashed and assigned to a bucket. Documents whose bands fall into the same bucket are considered potential matches for a more detailed comparison.[11] The probability that two documents with Jaccard similarity

s are grouped into the same bucket is given by the formula:

$1-(1-sr)b$

This approach effectively filters out a vast majority of dissimilar documents, leaving only a smaller set of candidates for pairwise comparison.[11]

The rigorous and computationally expensive nature of this process is a non-trivial engineering challenge. For example, deduplicating a dataset the size of C4 (~300 GB) can require a machine with 96 CPU cores and over 600 GB of RAM.[10] The immense effort and resources dedicated to this task underscore its importance. This meticulous process ensures that the model learns from a diverse and non-redundant set of examples, forcing it to extract and generalize from underlying patterns rather than simply memorizing repeated information. The result is a more robust and capable model that is better able to generalize to new, unseen data, which directly enhances its final performance and intelligence.

Part II: Mathematical and Statistical Principles of LLM

Training

A. Quantifying Textual Significance: Vectorization and Corpus-Level Statistics

For a GPT model to process text, it must first be converted into a numerical format. This transformation, known as vectorization, is a cornerstone of natural language processing (NLP). While modern LLMs employ sophisticated neural-based methods, a historical and still highly relevant technique for quantifying textual significance is Term Frequency-Inverse Document Frequency (TF-IDF).[12] TF-IDF is a statistical measure used to evaluate the relevance of a word to a document in a collection of documents (a corpus).[12]

The TF-IDF score is the product of two distinct metrics: Term Frequency (TF) and Inverse Document Frequency (IDF). Term Frequency measures how often a word appears in a specific document. The simplest form of this is a raw count, but a more common method normalizes the count by the total number of words in the document:

TF(t,d)=total number of words in document dnumber of occurrences of term t in document d

Inverse Document Frequency, conversely, measures how common or rare a word is across the entire corpus. Its purpose is to diminish the weight of common words, such as "the" or "is," which appear in many documents and thus carry little discriminative power.[13] It is calculated as the logarithm of the total number of documents divided by the number of documents containing the term:

IDF(t,D)=log(number of documents with term t in corpus Dtotal number of documents N)

The final TF-IDF score is the product of these two values:

TF−IDF(t,d,D)=TF(t,d)×IDF(t,D)

A higher score indicates that a word is highly relevant to a specific document and is relatively rare across the corpus.[12] For example, in a corpus of three documents, a word like "the" might have an IDF of 0 because it appears in all of them, while a more specific term like "cat" would have a higher, non-zero IDF score, thus giving it a more significant TF-IDF value.[12]

Another powerful statistical method for analyzing text data is **cosine similarity**, which measures the similarity between two non-zero vectors in a multi-dimensional space.[14] In text

analysis, documents can be represented as vectors (e.g., using TF-IDF), and cosine similarity can be used to determine their semantic proximity, regardless of their length.[14] The formula for cosine similarity between two vectors, A and B, is defined as the dot product of the vectors divided by the product of their magnitudes:

similarity(A,B)=cos(θ)=||A|| ||B||A·B=Σi=1nAi2Σi=1nBi2Σi=1nAiBi

The resulting value ranges from -1 to 1, where 1 signifies that the vectors are identical in direction, 0 indicates orthogonality (no similarity), and -1 means they are diametrically opposed.[16] By focusing on the angle between vectors, cosine similarity provides a measure of semantic similarity that is independent of document length, which is crucial for comparing documents of varying sizes.[14]

These explicit statistical methods for quantifying and comparing text provided the foundational logic that paved the way for modern, neural-based representations. While modern LLMs do not rely on these exact formulas for their core processing, they learn the same underlying principles implicitly. The ability of an embedding to place semantically similar words close to each other in a high-dimensional space is a more nuanced, dynamic, and powerful evolution of the explicit, frequency-based logic of TF-IDF and cosine similarity. These classical statistical methods remain highly relevant today for data preprocessing and retrieval-augmented generation (RAG) pipelines, and they serve as a critical conceptual bridge from traditional NLP to the more complex neural architectures that define the current era of artificial intelligence.

B. Optimizing the Training Process: Batching, Shuffling, and Hyperparameters

Training a large language model on a massive dataset, such as The Pile or Common Crawl, is a computationally Herculean task. Feeding the entire dataset to the model at once for a single parameter update, a method known as "full-batch training," is not feasible due to the immense memory and computational requirements.[17] To circumvent this, deep learning models, including LLMs, employ a standard optimization method called

mini-batch gradient descent.[17]

This approach involves dividing the training data into smaller, manageable chunks called mini-batches. Each mini-batch contains a small subset of the total dataset, which is processed sequentially. For a dataset of 1,000 training examples, a batch size of 100 would result in 10 mini-batches.[17] The model calculates a loss and updates its internal parameters after processing each mini-batch, rather than waiting for the entire dataset. This process is

repeated for each mini-batch until all data has been processed, which constitutes a single "epoch".[17] The mini-batch approach offers significant advantages: it is memory-efficient, enables frequent model updates, and makes training large-scale models practical.[17]

A complementary and equally critical practice is data **shuffling**. At the beginning of each training epoch, the entire dataset is randomly shuffled.[17] This randomization is crucial for several reasons. It prevents the model from learning the specific order of the training data, which could lead to memorization and a failure to generalize. Shuffling also ensures that the model is exposed to a diverse and random mix of data in each mini-batch, which helps to mitigate biases that might be present in a non-random data distribution and promotes more stable and effective learning.[17]

These practices, along with the careful tuning of hyperparameters such as learning rate and number of epochs, are essential for managing the computational load and ensuring the efficacy of the training process.[7]

C. Evaluating Model Performance: Perplexity and Beyond

The performance of an LLM is not measured by a single metric but by a suite of statistical and task-specific evaluations. As previously discussed in the context of data filtering, **perplexity** is a fundamental metric for evaluating a language model's performance.[9] It quantifies the model's confidence in its predictions, with a lower score indicating a more accurate and confident model.[9] The mathematical formula for perplexity, for a given text sequence

$W=(w_1,w_2,\ldots,w_N)$, is defined as the inverse of the geometric mean of the token probabilities, normalized by the number of tokens:

$$\text{Perplexity}(W)=P(w_1,w_2,\ldots,w_N)^{-\frac{1}{N}}=\frac{1}{\sqrt[N]{P(w_1,w_2,\ldots,w_N)}}=\frac{1}{\sqrt[N]{\prod_{i=1}^{N}P(w_i \mid w_1,\ldots,w_{i-1})}}$$

This formula shows that perplexity is directly tied to the model's prediction capabilities. A lower perplexity score indicates that the model assigns a higher probability to the text sequence, meaning it finds the sequence less "perplexing" or surprising.[9]

While perplexity is a general measure of a model's linguistic fluency and uncertainty, other statistical metrics are used to evaluate performance on specific tasks. For example, **BLEU (Bilingual Evaluation Understudy)** is a widely used metric for evaluating the quality of machine-translated text by comparing the model's output to a set of human-created reference translations.[19]

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a collection of metrics

designed for evaluating summarization tasks, focusing on the overlap of n-grams between the model's generated summary and a reference summary.[19] The use of multiple metrics is necessary because no single metric can capture every facet of an LLM's complex behavior, which includes accuracy, relevance, and the absence of harmful content.[19]

Part III: The Lifecycle of a Prompt: From User Input to Tokenized Vector

A. The Foundation of Communication: The Tokenization Pipeline

The journey of a user's prompt into a GPT model begins with a crucial preparatory step: tokenization. A large language model does not process raw text; it processes numerical representations. Tokenization is the process of breaking down an input text into smaller, discrete units called tokens, and then converting these tokens into a sequence of integer IDs.[20] A token can be a full word, a subword, or even a single character.[22] This initial transformation is fundamental because it converts unstructured human language into a structured format that a machine can understand and process.[23]

The most prevalent tokenization methods for modern LLMs are based on subword algorithms. These methods are designed to handle the inevitable problem of out-of-vocabulary (OOV) words, which are words that the model has never encountered during training. By breaking down words into smaller, semantically meaningful units, a model can process and understand new or rare words, such as proper nouns or complex compound words, without having them explicitly in its vocabulary.[24]

Two of the most widely used subword tokenization algorithms are Byte-Pair Encoding (BPE) and WordPiece. The BPE algorithm, which was originally developed as a data compression technique, builds a vocabulary by iteratively merging the most frequently occurring adjacent pairs of characters or token sequences in a training corpus.[25] The process is best illustrated with an example.

Image: BPE Tokenization Example
A sequential diagram demonstrating the BPE algorithm.

1. **Initial Text:** "the cat in the hat"
2. **Character-level Tokens:** "t h e c a t i n t h e h a t" (A list of individual characters)

3. **Merge 1:** The most frequent pair is "th". It appears twice. The algorithm merges "th" into a new token.
    - Resulting tokens: "th e c a t i n th e h a t"
4. **Merge 2:** The next most frequent pair is "th e". It appears twice. The algorithm merges "th e" into a new token.
    - Resulting tokens: "the c a t i n the h a t"
5. **Merge 3:** The next most frequent pair is "the ". It appears twice. The algorithm merges "the " into a new token.
    - Resulting tokens: "the cat i n the hat" (The diagram shows the words are now tokens because of the spaces.)

This process continues until a predefined vocabulary size is reached. The algorithm learns a set of merge rules that can be applied to any new text, effectively compressing and segmenting it into a sequence of learned subwords.[27]

The WordPiece algorithm is similar to BPE but introduces a critical distinction in its training philosophy.[29] Instead of simply merging the most frequent pair, WordPiece uses a scoring formula to prioritize which pairs to merge. The score is calculated as:

score=freq_of_first_element×freq_of_second_elementfreq_of_pair

This formula does not simply favor the most common pair but rather rewards pairs whose individual components are less frequent in the vocabulary. The algorithm explicitly seeks to merge subwords that are highly co-dependent but not independently common, thereby creating more linguistically meaningful subwords.[29] This divergence reveals a fundamental choice in tokenizer design: BPE optimizes for compression and efficiency, while WordPiece aims to discover more meaningful subword units based on a calculated co-dependency. This strategic difference influences how the model learns to represent language, demonstrating that tokenization is not a neutral process but a design decision that directly impacts a model's behavior and performance.[29]

B. The Vectorial Mind: From Tokens to Contextual Embeddings

After the input text is tokenized into a sequence of integer IDs, the next step is to convert these IDs into meaningful numerical vectors that the Transformer architecture can process. This is the role of the **embedding layer**, which can be conceptualized as a vast lookup table or matrix.[30] Each row in this matrix corresponds to a unique token ID in the model's vocabulary, and the row itself is a high-dimensional vector. When a token ID is passed to the embedding layer, it simply retrieves the corresponding vector from this matrix.[31] These vectors are not random; they are learned during the model's training and are designed to capture the

semantic properties of the tokens. Words with similar meanings, such as "university" and "college," will have vectors that are closer to each other in this high-dimensional space than they are to unrelated words like "banana".[20]

A central challenge of the Transformer architecture is its parallel processing nature. Unlike previous models that processed text sequentially, Transformers analyze entire sequences simultaneously, which eliminates their inherent ability to understand the order of words.[34] This is where

positional encoding becomes indispensable. Positional encoding is a technique that injects information about the position of each token into the model's input vector.[35] The original "Attention Is All You Need" paper introduced a sinusoidal method for this purpose, using a unique combination of sine and cosine functions to generate a deterministic encoding for each position in the sequence.[34]

The formulas for this positional encoding are as follows:

For even-indexed dimensions (2i):
PEpos,2i=sin(10000dmodel2ipos)
For odd-indexed dimensions (2i+1):
PEpos,2i+1=cos(10000dmodel2ipos)
Where pos is the position of the token in the sequence, i is the dimension index, and dmodel is the dimensionality of the model's embedding vectors.[35]

The final input vector that is fed into the Transformer is a fusion of both semantic and positional information. This is accomplished by performing an element-wise addition of the token embedding vector and its corresponding positional encoding vector.[32] This combined vector is what allows the subsequent self-attention mechanism to function effectively. Without the positional information, the model would be unable to differentiate between two identical words at different locations in a sentence, crippling its ability to understand grammatical structure and context.[33] The precise combination of semantic meaning from the embedding layer and sequential context from the positional encoding is the causal trigger that enables the Transformer to process language with a profound, context-aware understanding.

The entire user input pipeline can be visualized as a sequential flow of operations:

Image: The User Input Pipeline Flowchart
A flowchart illustrating the real-time processing of a user's prompt. The process begins with User Text Input, which flows into a box labeled Tokenizer. Inside this box, the text is split into a sequence of Token IDs. These IDs are then fed into two parallel processes: a Token Embedding Lookup Table and a Positional Encoding Generator. The Token Embedding Lookup Table retrieves a Token Embedding Vector for each ID. The Positional Encoding Generator

produces a Positional Encoding Vector for each position. Both types of vectors are then combined in a box labeled Element-wise Addition. The final output is a sequence of Final Input Vectors which are ready to be processed by the Transformer model.

Part IV: Conclusion and Future Outlook

A. Synthesis of the Data-Centric Paradigm

The intelligence of a GPT model is not a singular phenomenon; it is a direct consequence of a multi-stage data pipeline, from vast data ingestion to the final transformation of a user's prompt. The model's ability to generate coherent and contextually relevant text is inextricably linked to the quality and diversity of its training data. The journey from raw, unfiltered web pages to a cleaned and normalized corpus is a meticulous process governed by statistical and computational methods, including perplexity-based filtering and MinHash-LSH deduplication. These practices, while often unseen, are fundamental to preventing overfitting and ensuring the model's capacity for generalization.

Similarly, the real-time processing of a user's query is a logical progression of transformations. Tokenization converts human language into machine-readable IDs, while the embedding layer and positional encodings transform these IDs into rich, context-aware vectors. The final input vector, a fusion of semantic and sequential information, is the critical element that empowers the Transformer's attention mechanism. The entire data life cycle, from training to inference, is a testament to the principle that a model's capabilities are profoundly defined by the quality and integrity of the data it processes.

B. The Path Forward: Scaling, Transparency, and Ethical Data Curation

As the scale of LLMs continues to grow, so do the challenges associated with their data. Research has indicated that there are indeed limits on the horizon for scaling language models indefinitely, as we may soon reach a point where there is insufficient existing data to support further learning.[37] This underscores a critical shift from simply acquiring more data to utilizing existing data more effectively.[37]

Furthermore, the ethical and social implications of data sourcing demand greater

transparency. As evidenced by the biases found in popular datasets, the lack of clear documentation on filtering methodologies and content origins creates a profound risk of perpetuating and amplifying societal biases.[5] The future of LLMs must be defined by a commitment to data transparency, with clear ethical frameworks for data collection, curation, and bias mitigation. This is not merely a technical concern but a moral imperative, as the very foundation of artificial intelligence is built upon the data we provide it.

C. Seminal Papers for Further Study

For those seeking to delve deeper into the foundational concepts discussed in this report, the following seminal research papers are recommended:

- **"Attention Is All You Need"** by Vaswani et al. (2017) [34]
- **"Improving Language Understanding by Generative Pre-Training"** by Radford et al. (2018) [39]
- **"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"** by Devlin et al. (2018) [39]
- **"Language Models are Unsupervised Multitask Learners"** by Radford et al. (2019) [39]
- **"A New Algorithm for Data Compression"** by Philip Gage (1994) [25]
- **"SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing"** by Kudo and Richardson (2018) [40]
- **"The Pile: An 800GB Dataset of Diverse Text for Language Modeling"** by Gao et al. (2020) [4]
- **"Scaling Data-Constrained Language Models"** (Cornell University, 2023) [37]

Works cited

1. Common Crawl - Open Repository of Web Crawl Data, accessed September 1, 2025, https://commoncrawl.org/
2. Mozilla Report: How Common Crawl's Data Infrastructure Shaped ..., accessed September 1, 2025, https://www.mozillafoundation.org/en/blog/Mozilla-Report-How-Common-Crawl-Data-Infrastructure-Shaped-the-Battle-Royale-over-Generative-AI/
3. The Pile (dataset) - Wikipedia, accessed September 1, 2025, https://en.wikipedia.org/wiki/The_Pile_(dataset)
4. The Pile, accessed September 1, 2025, https://pile.eleuther.ai/
5. C4 dataset - AIAAIC, accessed September 1, 2025, https://www.aiaaic.org/aiaaic-repository/ai-algorithmic-and-automation-incidents/c4-dataset
6. Mastering LLM Techniques: Text Data Processing | NVIDIA Technical Blog,

accessed September 1, 2025, https://developer.nvidia.com/blog/mastering-llm-techniques-data-preprocessing/
7. How To Train an LLM on Your Own Data in 6 Steps - TensorWave, accessed September 1, 2025, https://tensorwave.com/blog/how-to-train-an-llm-on-your-own-data
8. LLM Training Data: The 8 Main Public Data Sources - Oxylabs, accessed September 1, 2025, https://oxylabs.io/blog/llm-training-data
9. Perplexity as a determinant of text quality. - DEV Community, accessed September 1, 2025, https://dev.to/rotimi_ajigboye/perplexity-as-a-determinant-of-text-quality-2i5b
10. google-research/deduplicate-text-datasets - GitHub, accessed September 1, 2025, https://github.com/google-research/deduplicate-text-datasets
11. MINHASH_LSH | Milvus Documentation, accessed September 1, 2025, https://milvus.io/docs/minhash-lsh.md
12. Two minutes NLP — Learn TF-IDF with easy examples | by Fabio ..., accessed September 1, 2025, https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3
13. Understanding TF-IDF for Machine Learning | Capital One, accessed September 1, 2025, https://www.capitalone.com/tech/machine-learning/understanding-tf-idf/
14. Cosine similarity - Wikipedia, accessed September 1, 2025, https://en.wikipedia.org/wiki/Cosine_similarity
15. Cosine Similarity - LearnDataSci, accessed September 1, 2025, https://www.learndatasci.com/glossary/cosine-similarity/
16. Understanding the Cosine Similarity Formula - TiDB, accessed September 1, 2025, https://www.pingcap.com/article/understanding-the-cosine-similarity-formula/
17. Batching and Mini-Batch: Making Your Deep Learning Model Work Efficiently - Medium, accessed September 1, 2025, https://medium.com/@nasuhcanturker/batching-and-mini-batch-making-your-deep-learning-model-work-efficiently-1bb5d3481eda
18. trainingOptions - Options for training deep learning neural network - MATLAB - MathWorks, accessed September 1, 2025, https://www.mathworks.com/help/deeplearning/ref/trainingoptions.html
19. Statistical Methods for Evaluating LLM Performance - MachineLearningMastery.com, accessed September 1, 2025, https://machinelearningmastery.com/statistical-methods-for-evaluating-llm-performance/
20. Inside Large Language Models: Revealing How LLM Technology Actually Works, accessed September 1, 2025, https://tetr.com/blog/how-llm-technology-actually-works
21. Tokenization – A Complete Guide – Stacks As a Service - STAAS, accessed September 1, 2025, https://cms.staas.io/blog/tutorial/tokenization-a-complete-guide/
22. The Invisible Building Blocks of AI: What You Need to Know About Tokenization, accessed September 1, 2025,

https://medium.com/data-science-collective/the-invisible-building-blocks-of-ai-what-you-need-to-know-about-tokenization-acadd86a63ba
23. confused about embeddings and tokenization in LLMs : r/learnmachinelearning - Reddit, accessed September 1, 2025, https://www.reddit.com/r/learnmachinelearning/comments/1cs29kn/confused_about_embeddings_and_tokenization_in_llms/
24. What is WordPiece? - H2O.ai, accessed September 1, 2025, https://h2o.ai/wiki/wordpiece/
25. Byte-pair encoding - Wikipedia, accessed September 1, 2025, https://en.wikipedia.org/wiki/Byte-pair_encoding
26. Byte-Pair Encoding For Beginners | Towards Data Science, accessed September 1, 2025, https://towardsdatascience.com/byte-pair-encoding-for-beginners-708d4472c0c7/
27. Understanding Byte-Pair Encoding - Medium, accessed September 1, 2025, https://medium.com/@hsinhungw/understanding-byte-pair-encoding-fd196ebfe93f
28. Implementing A Byte Pair Encoding (BPE) Tokenizer From Scratch, accessed September 1, 2025, https://sebastianraschka.com/blog/2025/bpe-from-scratch.html
29. WordPiece tokenization - Hugging Face LLM Course, accessed September 1, 2025, https://huggingface.co/learn/llm-course/chapter6/6
30. Transformer (deep learning architecture) - Wikipedia, accessed September 1, 2025, https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)
31. Lecture 10: What are token embeddings? - YouTube, accessed September 1, 2025, https://www.youtube.com/watch?v=ghCSGRgVB_o
32. LLM Transformer Model Visually Explained - Polo Club of Data Science, accessed September 1, 2025, https://poloclub.github.io/transformer-explainer/
33. Understanding Embeddings in the Transformer Architecture - Gale Force AI, accessed September 1, 2025, https://galeforce.ai/ai-academy/f/understanding-embeddings-in-the-transformer-architecture
34. Attention Is All You Need - Wikipedia, accessed September 1, 2025, https://en.wikipedia.org/wiki/Attention_Is_All_You_Need
35. Positional Encoding in Transformers - GeeksforGeeks, accessed September 1, 2025, https://www.geeksforgeeks.org/nlp/positional-encoding-in-transformers/
36. What is Positional Encoding? - IBM, accessed September 1, 2025, https://www.ibm.com/think/topics/positional-encoding
37. Award-Winning Paper Unravels Challenges of Scaling Language Models - Cornell Tech, accessed September 1, 2025, https://tech.cornell.edu/news/award-winning-paper-unravels-challenges-of-scaling-language-models/
38. How To Scale Your Model - GitHub Pages, accessed September 1, 2025, https://jax-ml.github.io/scaling-book/
39. Seminal Papers about Large Language Models – TensorLoops, accessed

September 1, 2025, https://www.tensorloops.it/2024/01/06/seminal-papers-about-large-language-models/
40. What is SentencePiece? | Activeloop Glossary, accessed September 1, 2025, https://www.activeloop.ai/resources/glossary/sentence-piece/